

Estruturas de Dados para Malha de Triângulos - *Winged-Edge* e *Half-Edge*

Allan Yoshio Hasegawa

Orientador: Prof. R. S. U. Rosso Jr.



Malha de Triângulos (*triangle mesh*)

- Definição
 - Rede de triângulos que conecta uns aos outros por meio do compartilhamento de arestas e vértices para formar uma única superfície contínua
 - Uma lista de triângulos não conectados não é uma malha de triângulos
- É importante a gerencia eficiente da malha
 - Armazenamento em disco
 - Memória
 - Largura da Banda
 - Transmissão via rede ou dispositivos
 - Acesso a triângulos adjacentes
 - Subdivisão, modificação da malha, compressão, etc...



Malha de Triângulos (*triangle mesh*)

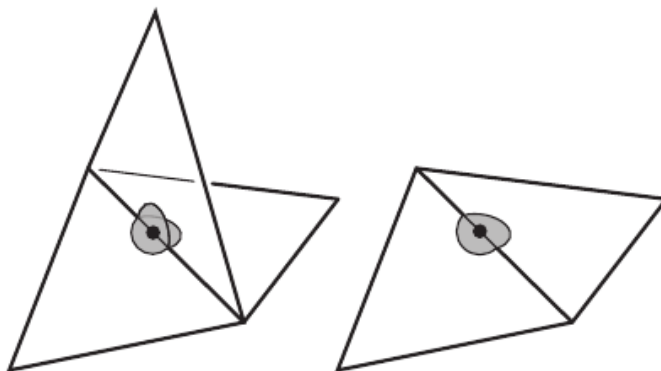
- Informações armazenadas
 - Vértices
 - Conjunto de triângulos (3 vértices)
 - Informações Opcionais
 - Geralmente armazenados em vértices e faces
 - Informações nos vértices geralmente são interpoladas na superfície
 - Material
 - Coordenadas da textura
 - Normal
 - Intensidade da luz
 - etc



Manifold

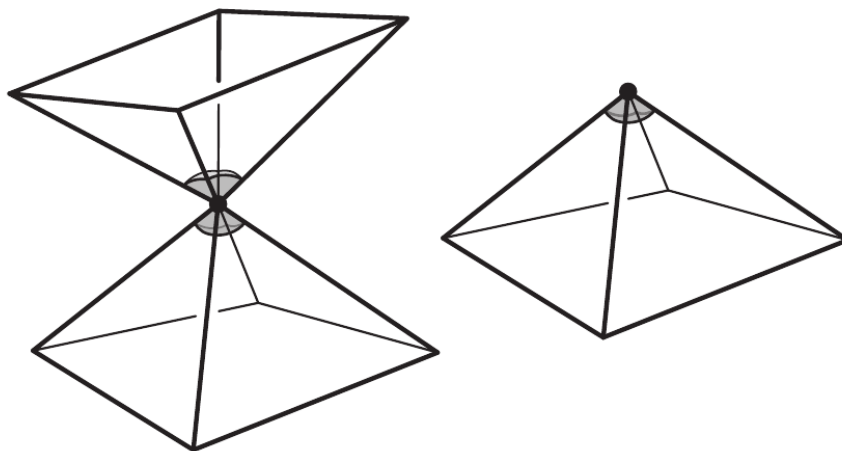
- Intuição
 - Superfície onde uma pequena vizinhança de qualquer ponto possa ser achatada em uma superfície plana
- Verificação de *manifold* (sem entrada/saída de água)
 - Todas as arestas são compartilhadas por exatamente dois triângulos
 - Todos os vértices têm um único, e completo, círculo de triângulos em sua volta





Arestas internas: não-*manifold* (esquerda) e *manifold* (direita)

Fonte: (Shirley *et al.*, 2009)



Vértices: não-*manifold* (esquerda) e *manifold* (direita)

Fonte: (Shirley *et al.*, 2009)



Manifold

- *Manifold* é útil para garantir que estruturas de dados para malhas de triângulos funcionem sem problemas
- *Manifold* não garante consistência na orientação das faces



Winged-Edge

- Estrutura de dados para armazenar malhas de triângulos
- Armazena informação da conectividade nas arestas



Winged-Edge

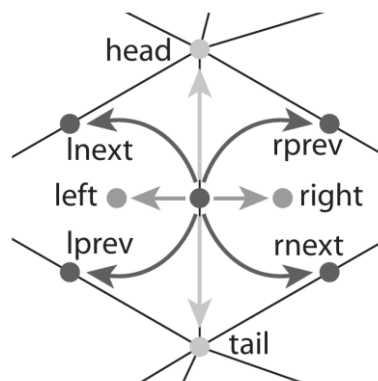
```

Edge {
    Edge lprev, lnext, rprev, rnext;
    Vertex head, tail;
    Face left, right;
}

Face {
    // ... per-face data ...
    Edge e; // any adjacent edge
}

Vertex {
    // ... per-vertex data ...
    Edge e; // any incident edge
}

```



Fonte: (Shirley et al. 2009)



Winged-Edge

- $O(1)$ para acessar as arestas de uma face ou vértice

```

EdgesOfVertex(v) {
    e = v.e;
    do {
        if (e.tail == v)
            e = e.lprev;
        else
            e = e.rprev;
    } while (e != v.e);
}

```

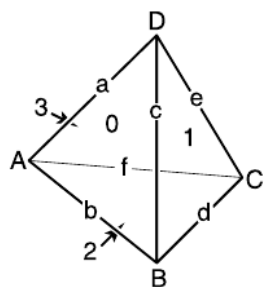
```

EdgesOfFace(f) {
    e = f.e;
    do {
        if (e.left == f)
            e = e.lnext;
        else
            e = e.rnext;
    } while (e != f.e);
}

```

Fonte: (Shirley et al. 2009)





face	edge
0	a
1	c
2	d
3	a

vertex	edge
A	a
B	d
C	d
D	e

edge	vertex 1	vertex 2	face left	face right	pred left	succ left	pred right	succ right
a	A	D	3	0	f	e	c	b
b	A	B	0	2	a	c	d	f
c	B	D	0	1	b	a	e	d
d	B	C	1	2	c	e	f	b
e	C	D	1	3	d	c	a	f
f	C	A	3	2	e	a	b	d

Exemplo de *Winged-Edge*

Fonte: (Shirley et al. 2009)



Half-Edge

- *Winged-Edge* sempre verifica qual a orientação da aresta antes de ir para a próxima
- *Half-Edge* soluciona esse problema armazenando duas metades de uma aresta para cada aresta
 - Uma metade de uma aresta para cada triângulo que compartilha uma aresta
 - As duas metades têm orientações opostas entre elas
 - Travessia é igual ao do *Winged-Edge*, mas sem verificação da orientação



Half-Edge

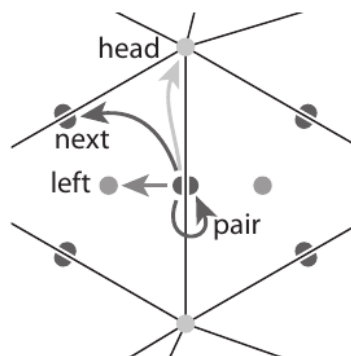
```

HEdge {
    HEdge pair, next;
    Vertex v;
    Face f;
}

Face {
    // ... per-face data ...
    HEdge h; // any h-edge of this face
}

Vertex {
    // ... per-vertex data ...
    HEdge h; // any h-edge pointing toward this vertex
}

```



Fonte: (Shirley et al. 2009)



Half-Edge

```

EdgesOfVertex(v) {
    h = v.h;
    do {
        h = h.pair.next;
    } while (h != v.h);
}

```

```

EdgesOfFace(f) {
    h = f.h;
    do {
        h = h.next;
    } while (h != f.h);
}

```

Fonte: (Shirley et al. 2009)



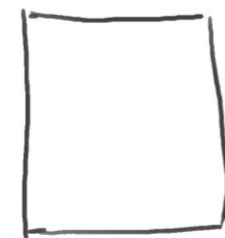
Estruturas de Dados Espaciais

Allan Yoshio Hasegawa

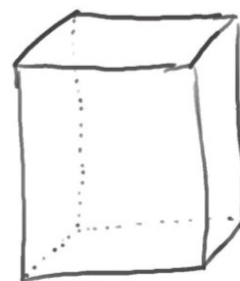


Voxel

- Analogia:
 - 2D: Pixel (Quadrado)*
 - Menor unidade divisível em uma imagem
 - Geralmente cada pixel representa uma intensidade de iluminação
 - 3D: Voxel (Cubo)*
 - Menor unidade de um modelo 3D
 - Um voxel pode armazenar diversas informações
 - Binário, normal, textura, função implícita, superfícies, densidade, etc...



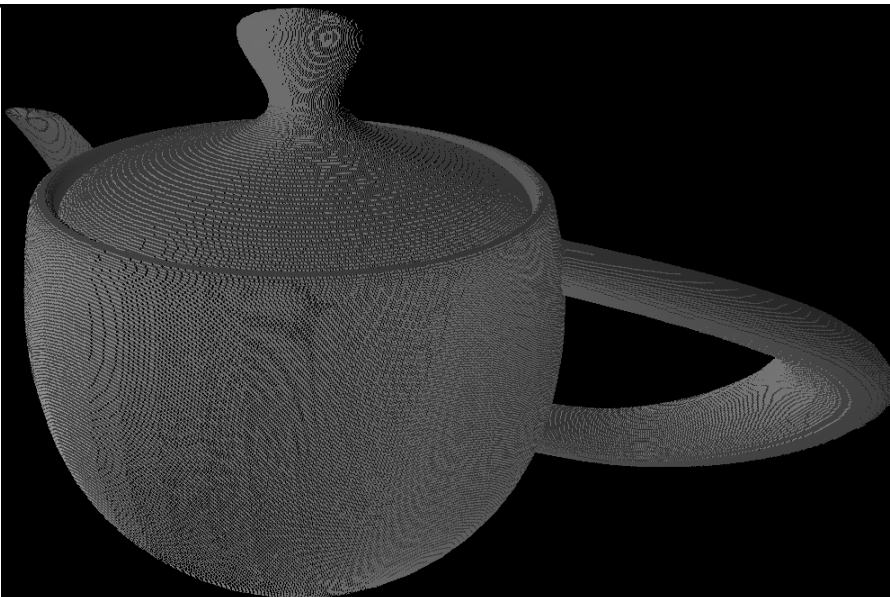
Pixel



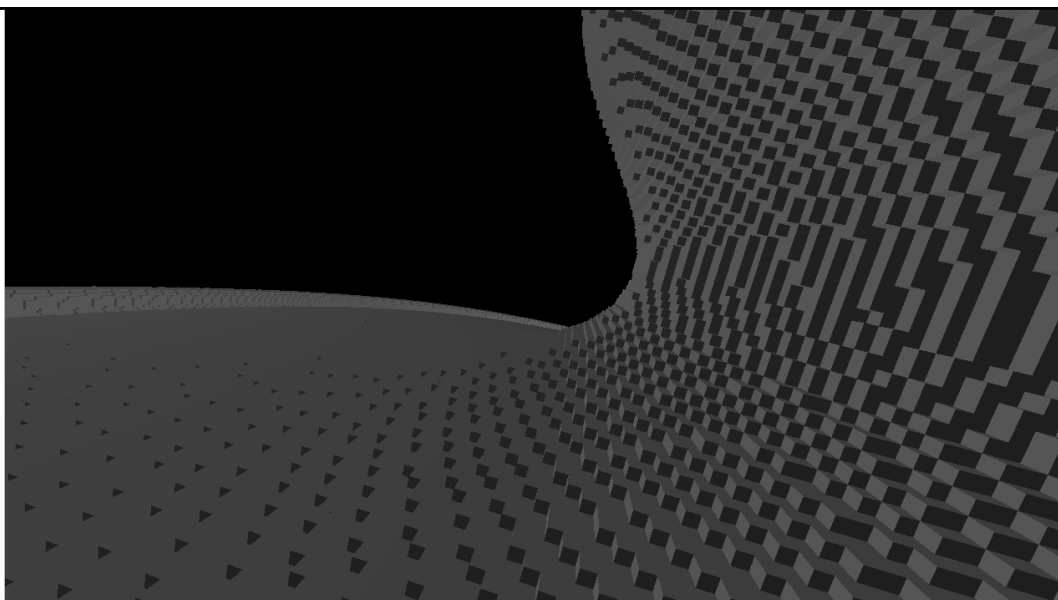
Voxel

* Simplificação para melhor entendimento. Um pixel pode não ser quadrado (Smith 1995).



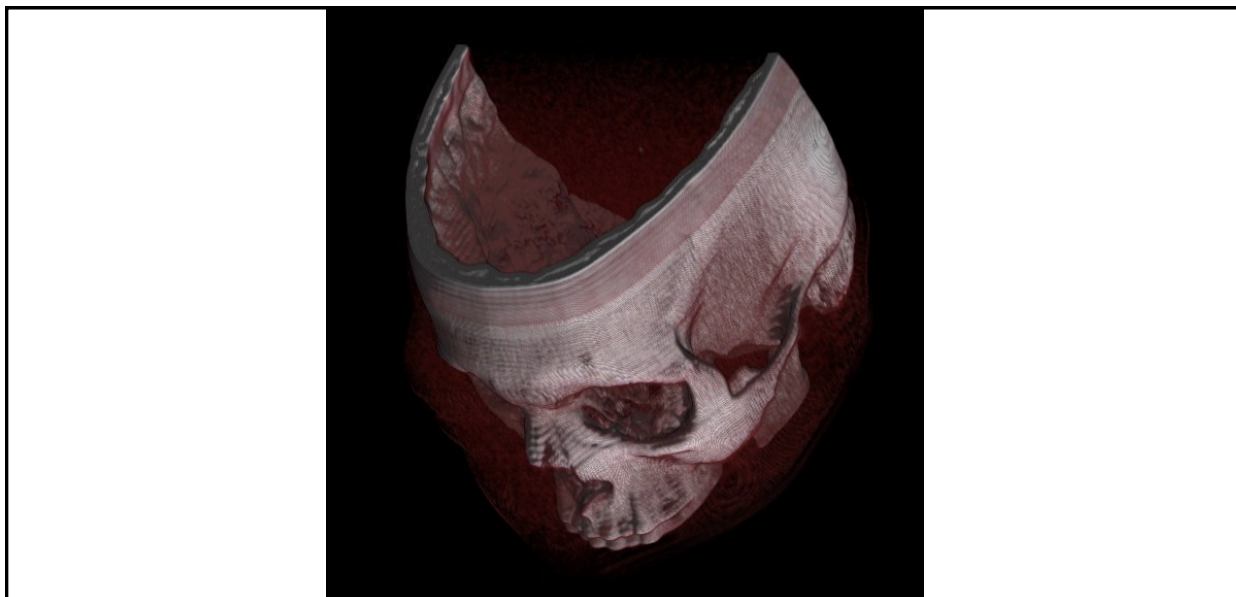


Voxelization do “Utah Teapot”. ~40 milhões de voxels.



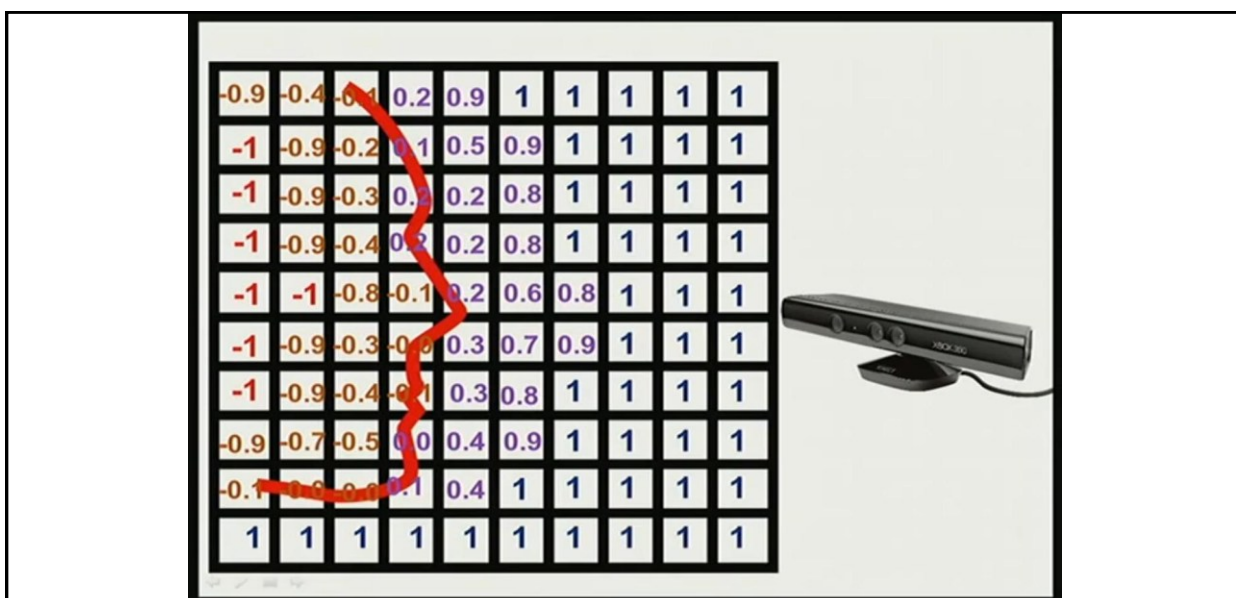
Visão de perto do modelo “Utah Teapot”.





Volume rendering. Voxels conseguem representar volume, não apenas a superfície.

Fonte: Volume Rendering. Disponível em: http://wikipedia.org/wiki/Volume_Rendering. Acesso em: 17/05/2014.



TSDf. Voxels informam a distância até a superfície.

Fonte: Documentation - Point Cloud Library (PCL). Disponível em: <http://pointclouds.org/documentation>. Acesso em: 17/05/2014.





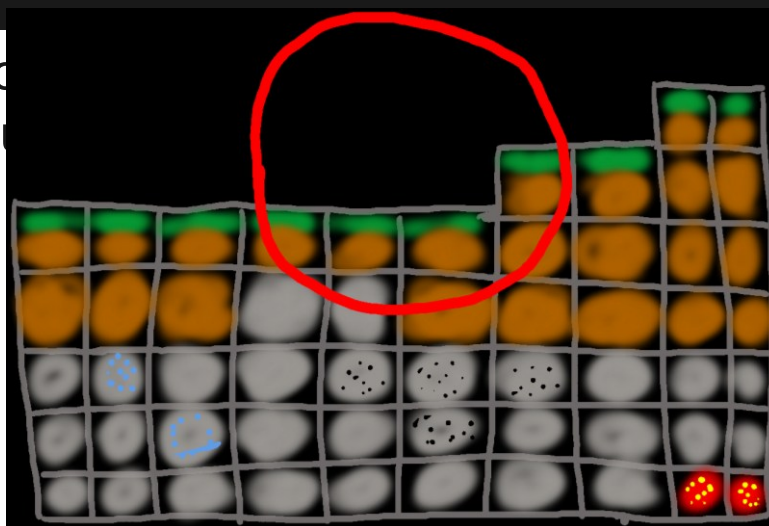
Minecraft. “Voxels” com textura, tipo, etc...

Fonte: Minecraft PS3 beginner's tips & tricks: how to survive, what to build first, & more. Disponível em: <http://www.officialplaystationmagazine.co.uk>. Acesso em: 17/05/2014.



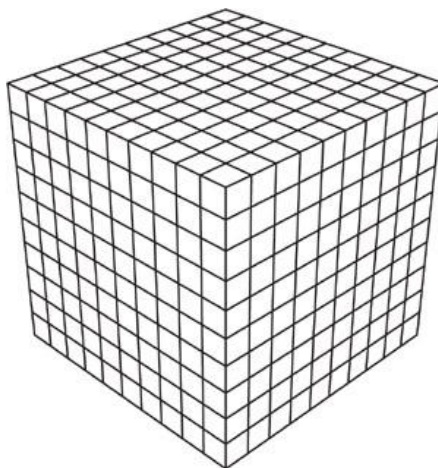
Problema

Como detectar o colisão entre jogador e voxel?



Lista (solução ingênua)

- *para todo voxel 'v':*
- *jogador colide com 'v'?*
- Sem informação espacial nos índices
- Não precisa armazenar voxels desnecessários
- Acesso em $O(n)$, onde 'n' é o número de voxels
- Complexidade $O(n*m)$ para teste de colisão entre dois modelos 3D em voxels, onde 'n' e 'm' são os números de voxels dos modelos.



Grid. Um *grid* uniforme 10x10x10.

Fonte: GPU Gems 3. Disponível em: http://http.developer.nvidia.com/GPUGems3/gpugems3_ch22.html. Acesso em: 17/05/2014.



Grid

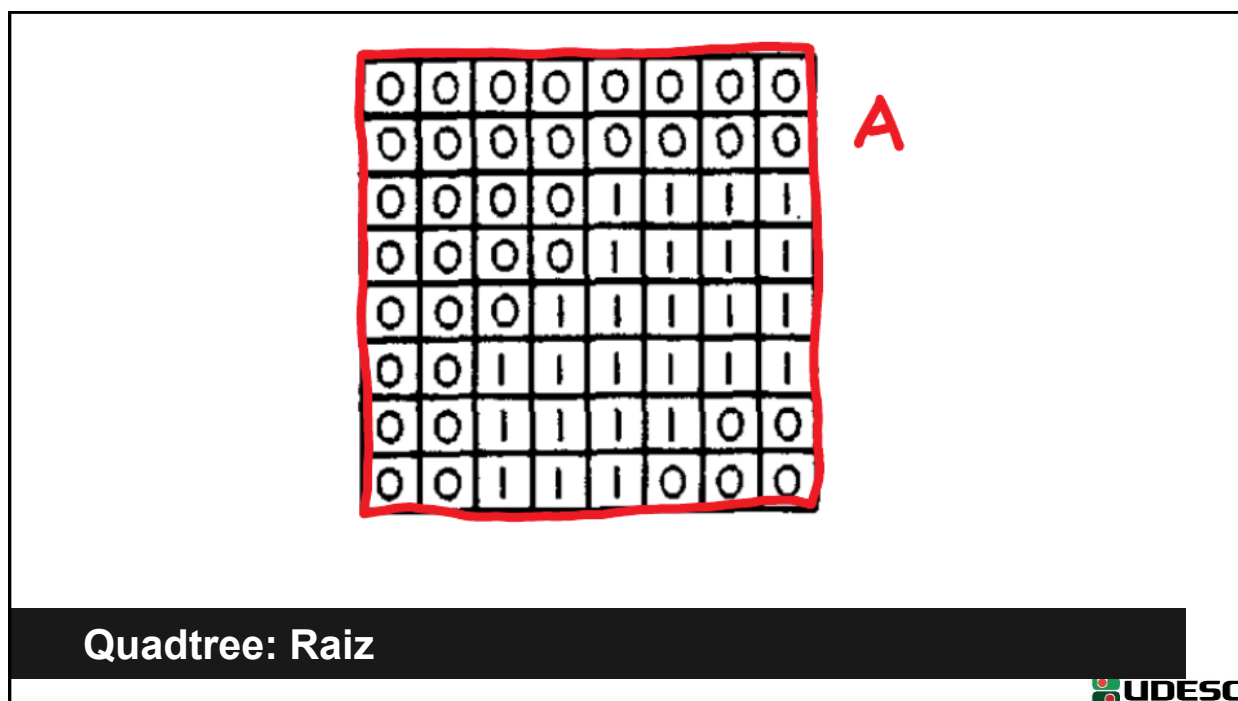
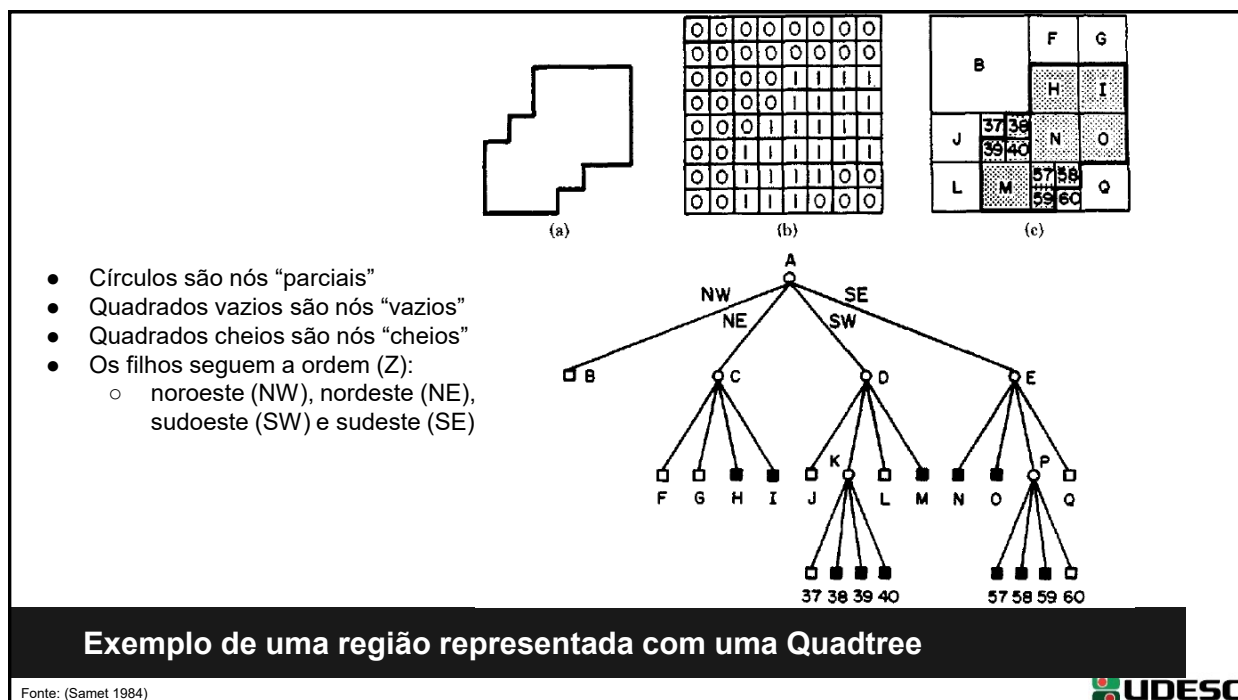
- Voxels armazenados em uma matriz 3D
- Acesso em $O(1)$ a um voxel usando as coordenadas dele
 - Acesso a vizinhança também é $O(1)$
 - Resolve o problema anterior, pois agora só precisa acessar os voxels vizinhos a posição do jogador
- Fácil implementação
- Alto consumo de memória
 - Grid de tamanho $10^3 = 1.000$ voxels
 - Grid de tamanho $20^3 = 8.000$ voxels ($8x$ ou 2^3x)
 - Porém ainda é possível ter grids $20x10x10$ (2.000 voxels)
 - Espaços homogêneos (como ar) ocupam o mesmo espaço que espaços heterogêneos, mesmo quando não são usados
 - Grids uniformes não se ajustam a diferentes complexidades de um modelo 3D
 - Consumo de memória é previsível

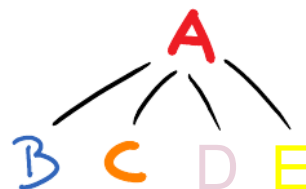
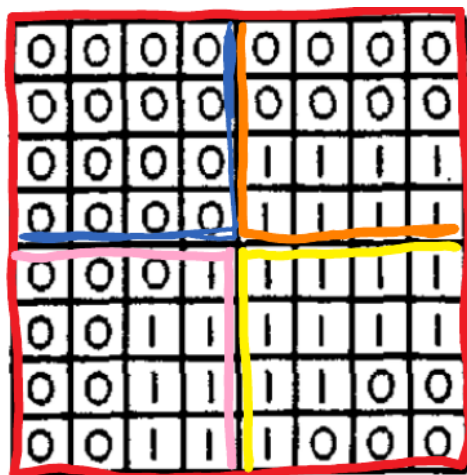


Quadtree e Octree

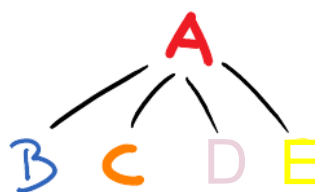
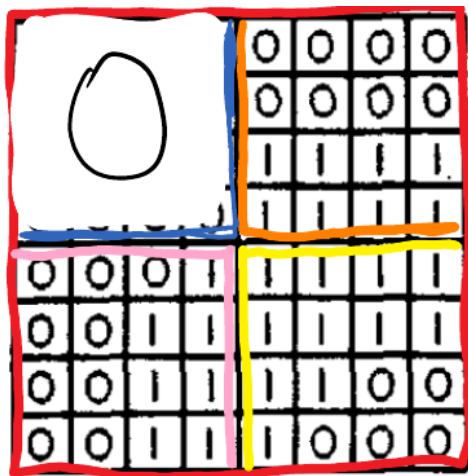
- A Quadtree foi desenvolvida principalmente para reduzir o consumo de memória por meio de agregação de dados homogêneos (Samet 1984)
- Quadtree para 2D e Octree para 3D
- Geralmente são representadas em forma de árvore
- Baseadas no princípio de decomposição recursiva
 - Cada nó, ao ser subdividido, cria 2^d filhos ($d = 2$ p/ Quadtree, $d = 3$ p/ Octree)
 - O tamanho do nó pai é igual a soma do tamanho dos nós filhos
 - Esse processo pode ser repetido recursivamente





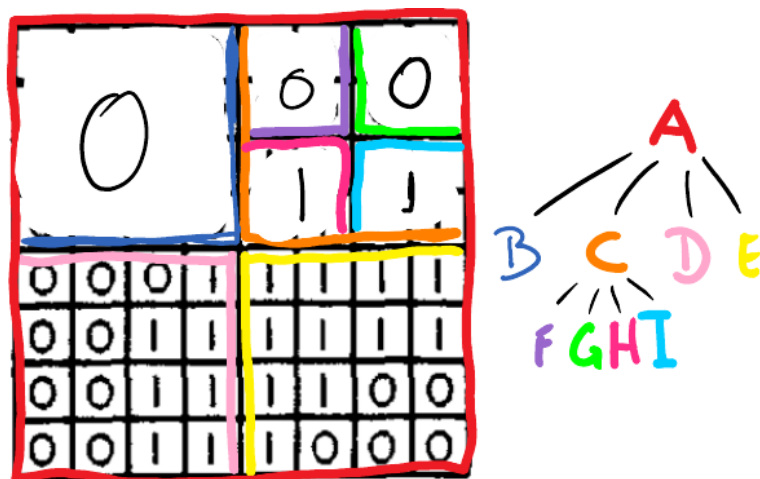


Quadtree: Subdivisão de A

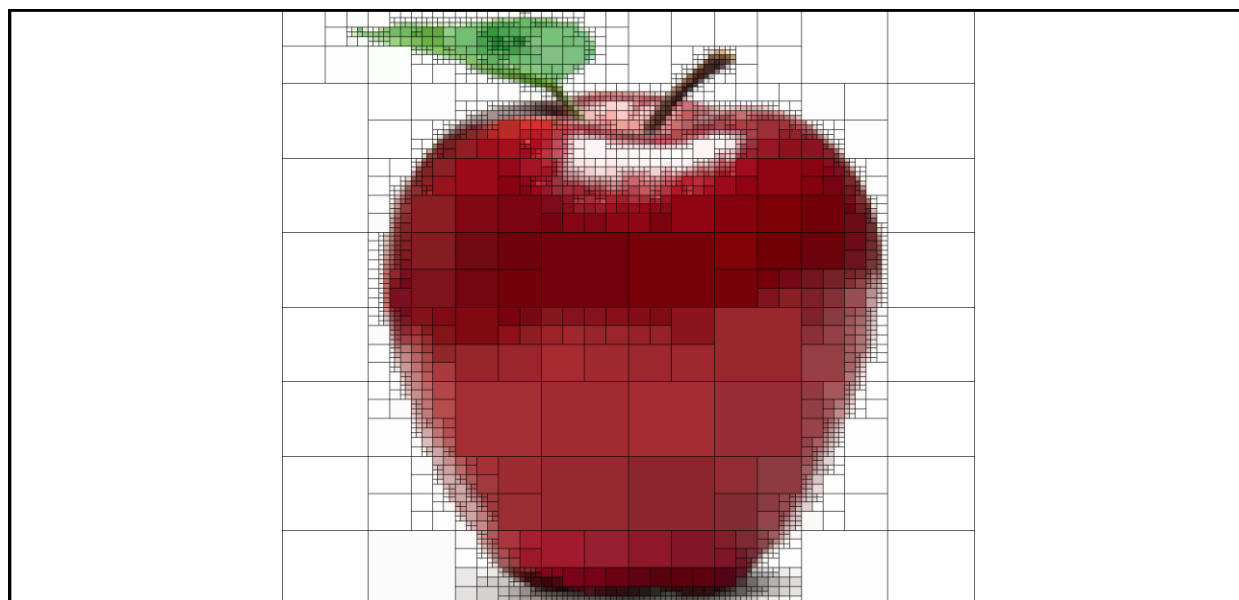


Quadtree: Verifica B





Quadtree: Subdivisão de C (Filhos F, G, H e I)



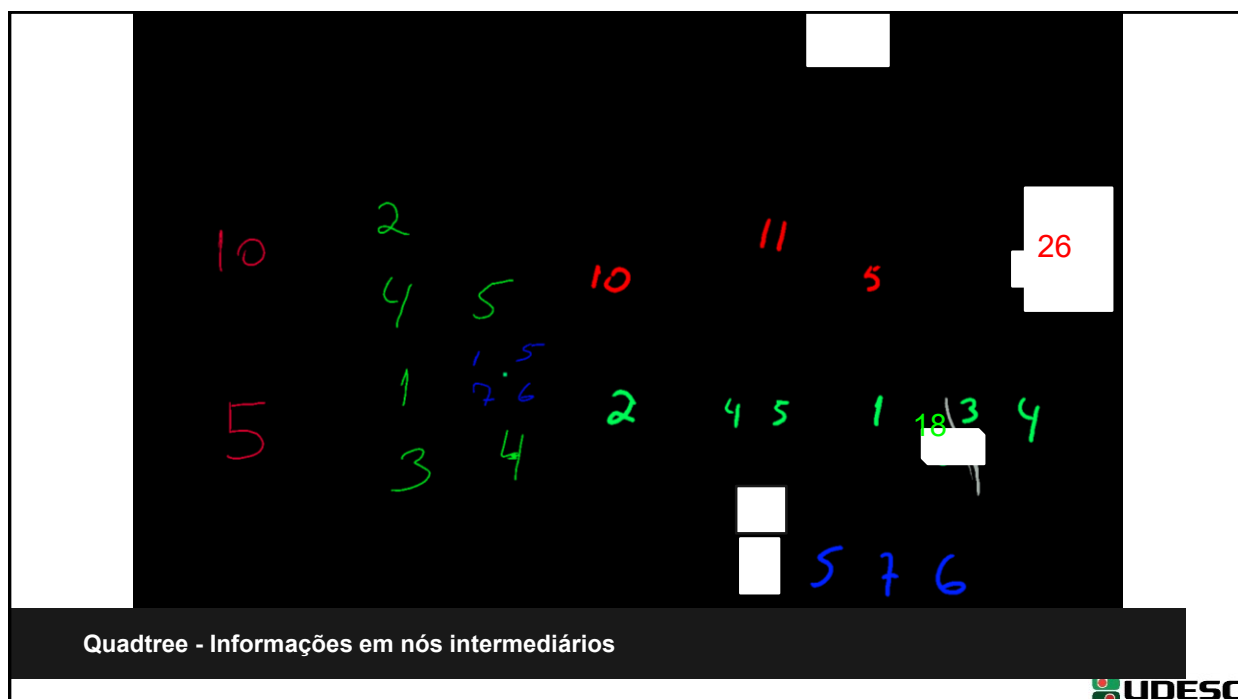
Exemplo de uma imagem armazenada em uma quadtree

Fonte: fogleman/Quads. Disponível em: <https://github.com/fogleman/Quads>. Acesso em 17/05/2014.



Quadtree e Octree

- Estruturas de dados hierárquicas
 - Nós pais têm a mesma área/volume que a soma dos nós filhos
 - Informações podem ser armazenadas nos nós intermediários (não apenas nas folhas)
 - “Encontre todas as cidades com uma população maior que 20 pessoas”
 - Nós folhas armazenam a população de uma área
 - Nós pais armazenam a soma da população dos filhos
 - Auxilia na aceleração de algoritmos



Quadtree e Octree

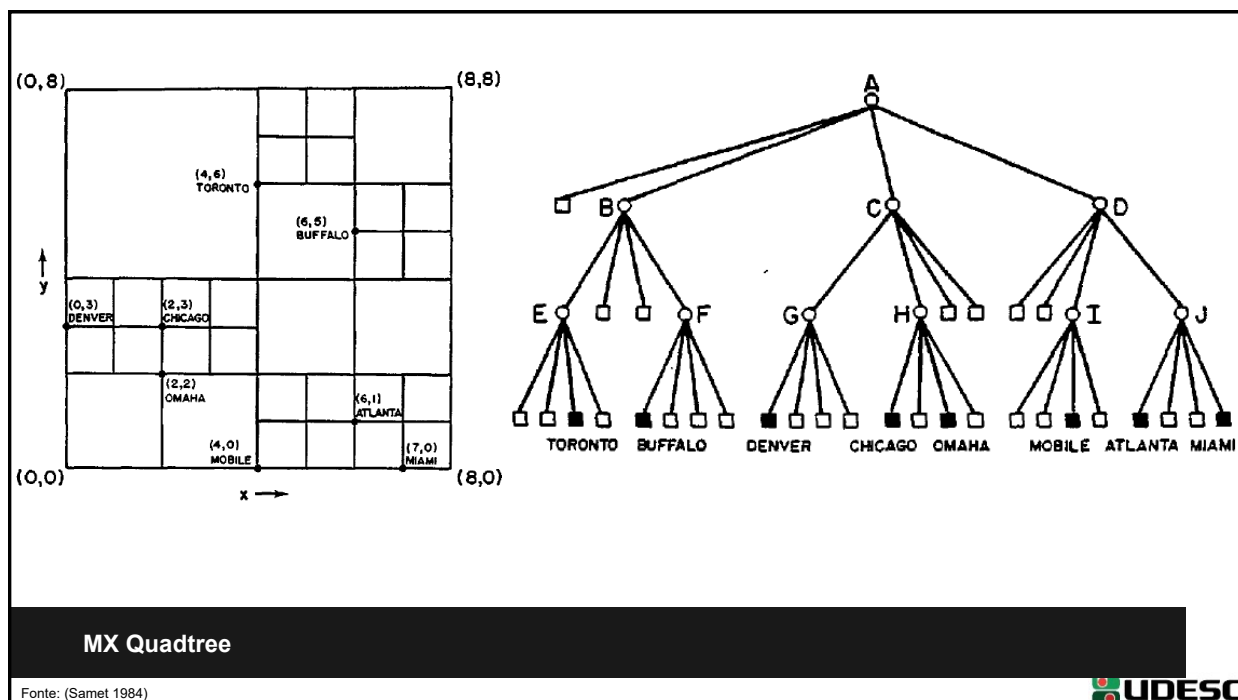
- Acesso em $O(\log n)$
 - 'n' é o número de elementos
 - $(\log n)$ é a altura da árvore
- Estrutura de Dados estática (Samet 1984)
 - É difícil mover voxels dentro da estrutura de dados
 - Imagine transladar todos os voxels para a direita
 - Cada voxel tem que ser removido, e depois inserido novamente
 - Um grid pode ser usado para realizar as transformações e depois a árvore é recriada



MX Quadtree

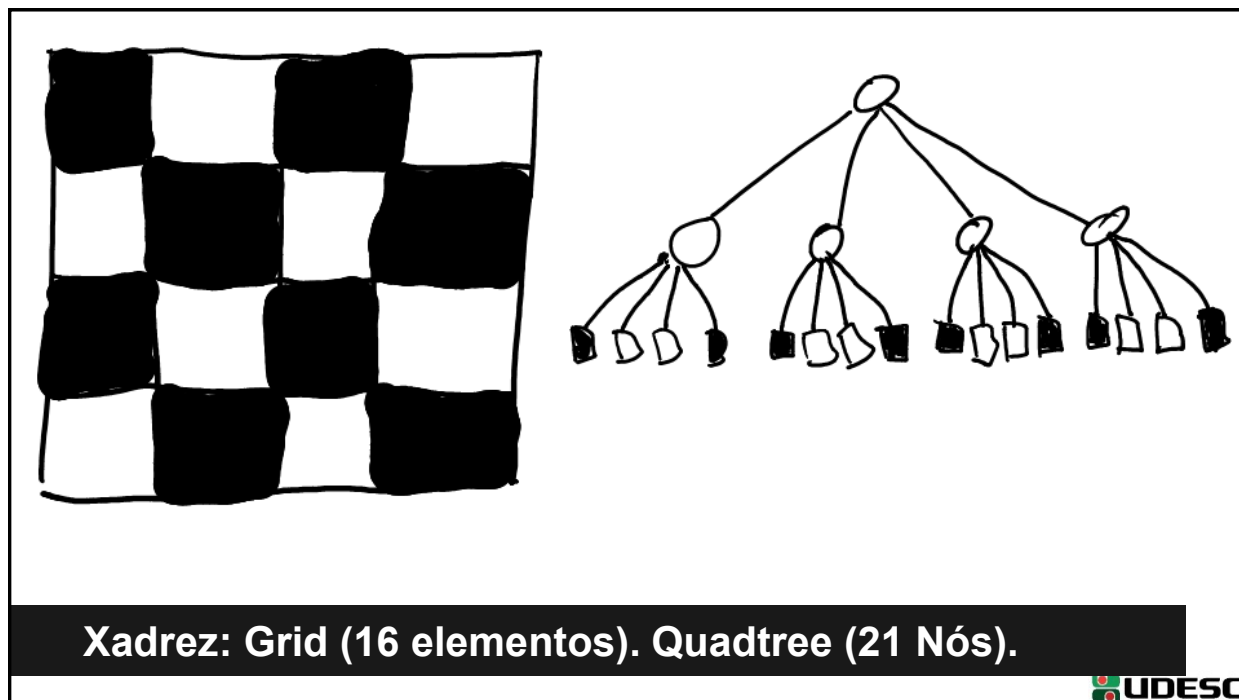
- MX vem de *Matrix*
- Precisa que os índices para acessar seus elementos sejam discretos
 - Elementos não zeros em uma matriz quadrada
- Cada nó é subdividido em 4 quadrantes de tamanhos iguais
- Cada folha têm apenas dois estados
 - Preto - Ponto de dado presente
 - Branco (ou vazio) - Ponto de dado não presente
- A posição de cada dado está relacionada a posição dele na árvore
 - Não é armazenada a posição nas folhas
 - Precisa seguir a convenção de NW, NE, SW e SE para saber a posição do dado





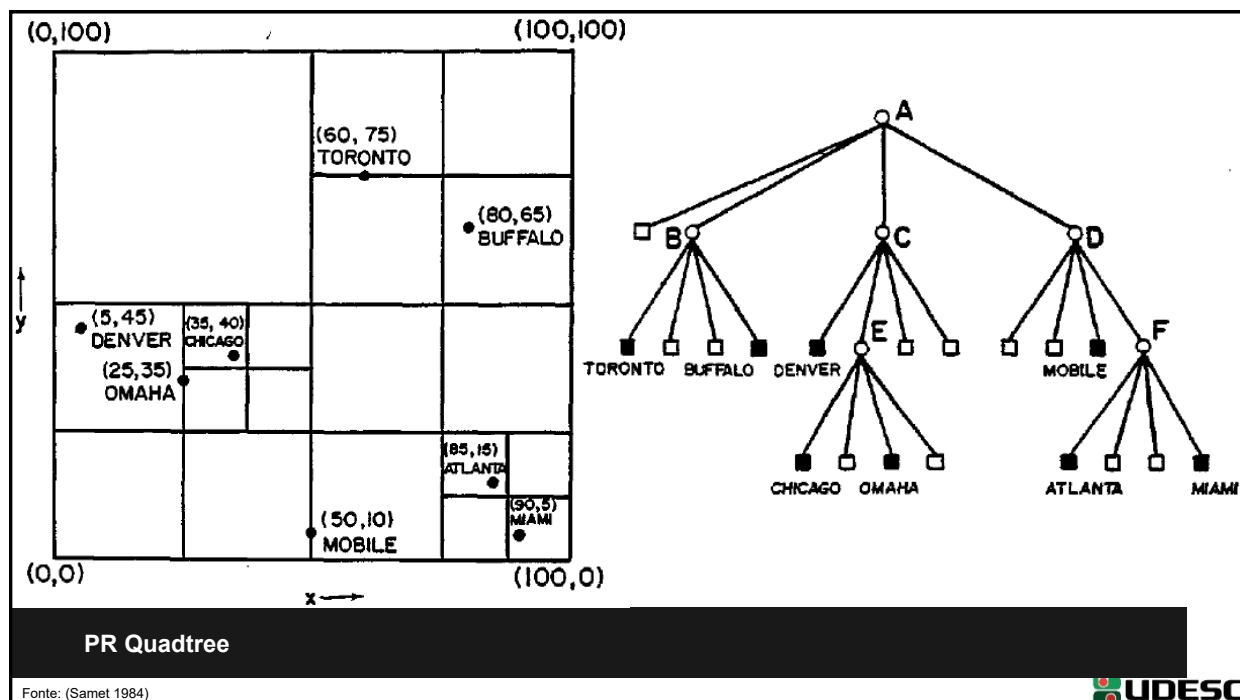
MX Quadtree

- Aumentar a precisão da localização do ponto é uma operação cara
 - Dobrar o tamanho da Quadtree custa 2^2 vezes o tamanho anterior de memória
 - É duplicada o tamanho nas duas dimensões
 - Dobrar o tamanho em uma Octree custa 2^3 vezes o tamanho anterior de memória
- Geralmente a Quadtree ocupa menos memória que um grid, porém em matrizes com muitos dados heterogêneos isso pode não acontecer
 - Exemplo: Tabuleiro de xadrez
- Geralmente usada em imagens e em representações discretas de funções contínuas



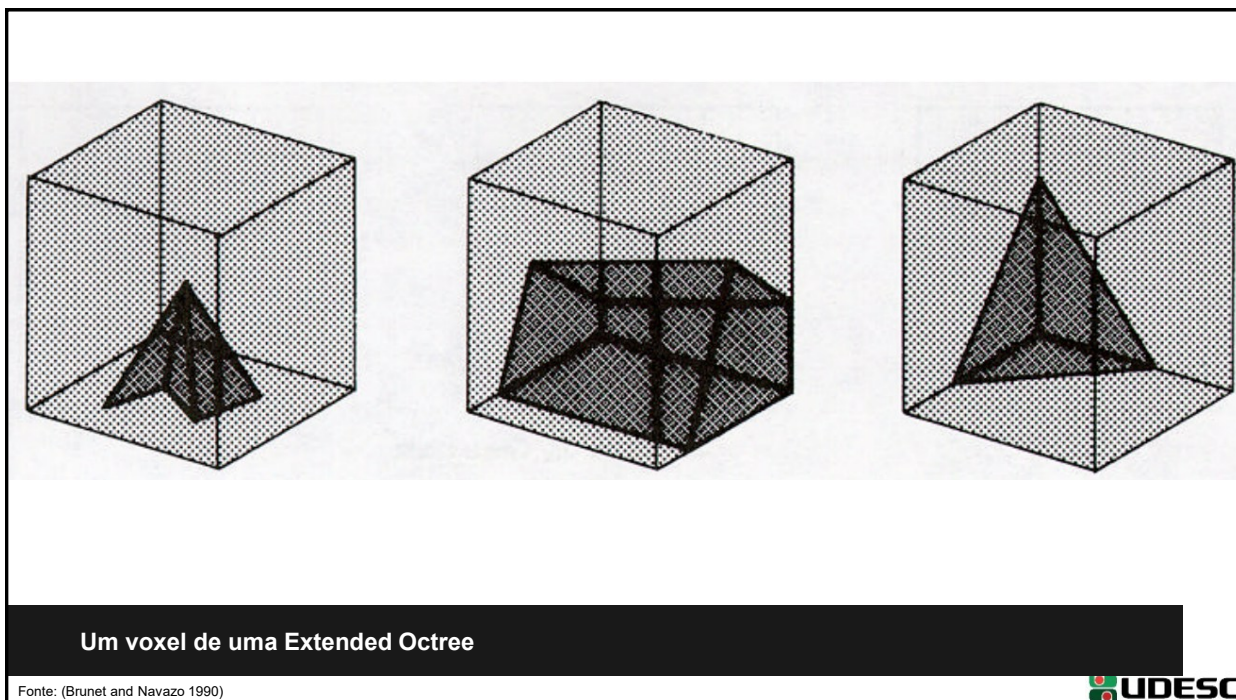
PR Quadtree

- PR Quadtree (*Point Region*)
- Organizada da mesma forma que a MX Quadtree
- Cada folha têm apenas dois estados:
 - Preto e Branco, porém folhas com dados também armazenam as suas coordenadas
- Uma folha contém no máximo um ponto
 - Caso uma folha já tenha um ponto: ao tentar inserir outro, o quadrante é subdividido até que cada ponto ocupe apenas uma folha
 - Essa operação pode causar muitas subdivisões caso a distância euclidiana entre os pontos seja pequena



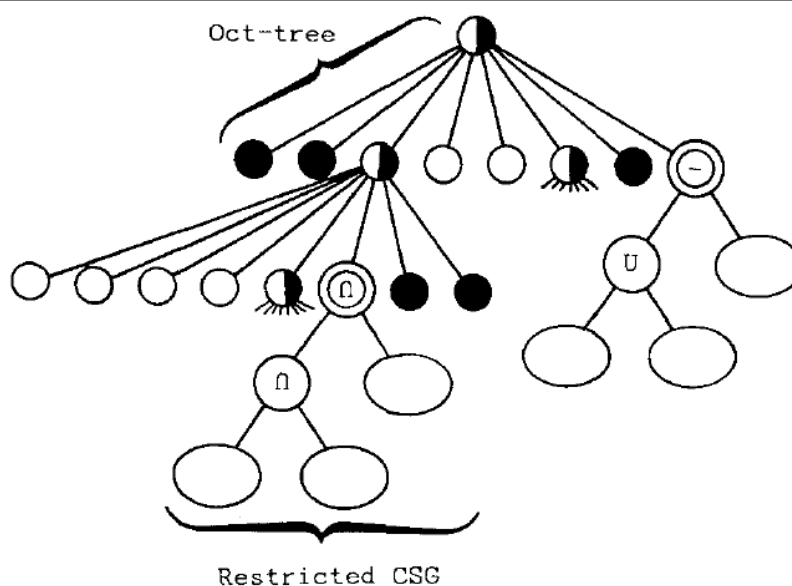
Extended Octree

- Representa a superfície de modelos sólidos usando polígonos
 - Polígonos são organizados nas folhas de uma Octree
- Resolve o problema onde modelos sólidos aproximados com “voxels binários” ocupam muita memória para uma boa aproximação da superfície
- Usa polígonos para melhor aproximar a superfície enquanto usa informações espaciais da Octree
 - Octree pode ser usada para acelerar algoritmos que executam operações locais (booleanas por exemplo)



Graftree

- Octree onde as folhas armazenam a raiz de uma árvore CSG
- Permite operações CSG locais
 - Octree é usado como uma estrutura de dados aceleradora
 - Útil para quando as operações booleanas afetam apenas regiões pequenas do modelo 3D
- Estados possíveis de um nó:
 - *FULL* - célula homogênea dentro do modelo
 - *EMPTY* - célula homogênea fora do modelo
 - *BOUNDARY* - raiz da CSG



Exemplo Graftree

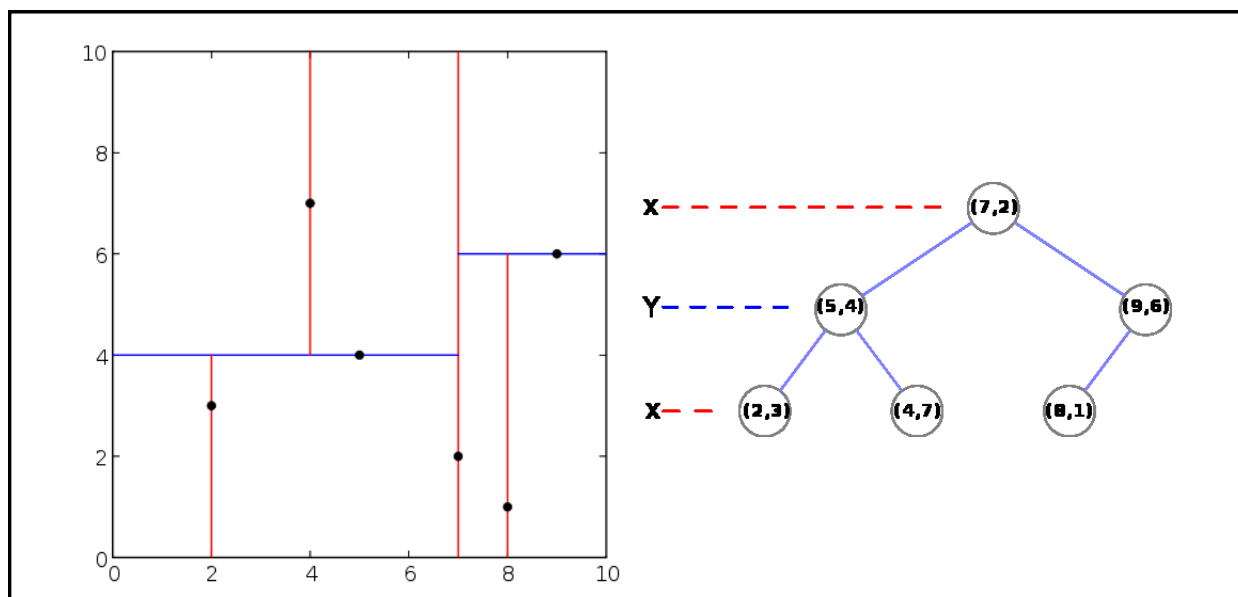
Fonte: (Kawashima et al. 1991)



KD-Tree

- Estrutura de dados espacial para organização de pontos
- KD-Tree é um tipo especial de árvores *Binary Space Partitioning* (BSP)
 - Cada nó de uma KD-Tree subdivide o espaço em duas regiões, assim um nó pai tem dois filhos (como em uma árvore binária)
 - Cada nó está associado a uma das k -dimensões, e o plano que divide as regiões é perpendicular ao eixo dessa dimensão
- KD-tree suporta qualquer número de dimensões
- Cada nível da árvore divide a região em uma dimensão
- KD-Trees são geralmente usadas em cenas com bastante espaçamento entre objetos





Exemplo de uma KD-Tree

Fonte: k-d tree. Disponível em: http://en.wikipedia.org/wiki/K-d_tree. Acesso em: 17/05/2014.



KD-Tree

- Ordem de inserção dos pontos na KD-Tree pode influenciar o seu balanceamento
- Existem diferentes técnicas para seleção do ponto onde é feita a subdivisão



KD - Tree

Complexidade em tempo

Operação	Na média	Pior caso
Busca	$O(\log n)$	$O(n)$
Inserção	$O(\log n)$	$O(n)$
Remove	$O(\log n)$	$O(n)$

Complexidade em Espaço

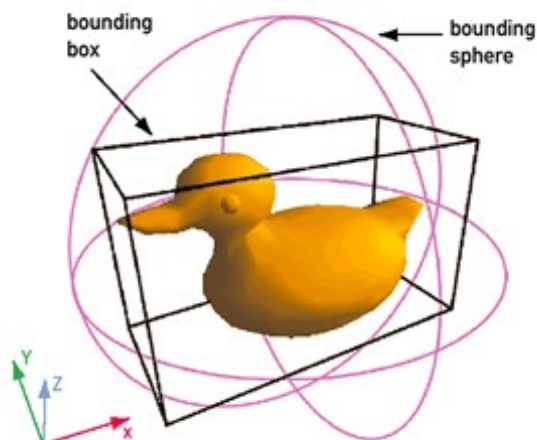
Espaço	$O(n)$	$O(n)$
--------	--------	--------



BVH

- *Bounding Volume (BV)*
 - É um volume que contém completamente a união de todos os objetos em um conjunto
 - Geralmente são usadas formas simples como esferas e caixas alinhadas aos eixos (AABB)





Bounding Volumes: bounding box e esfera

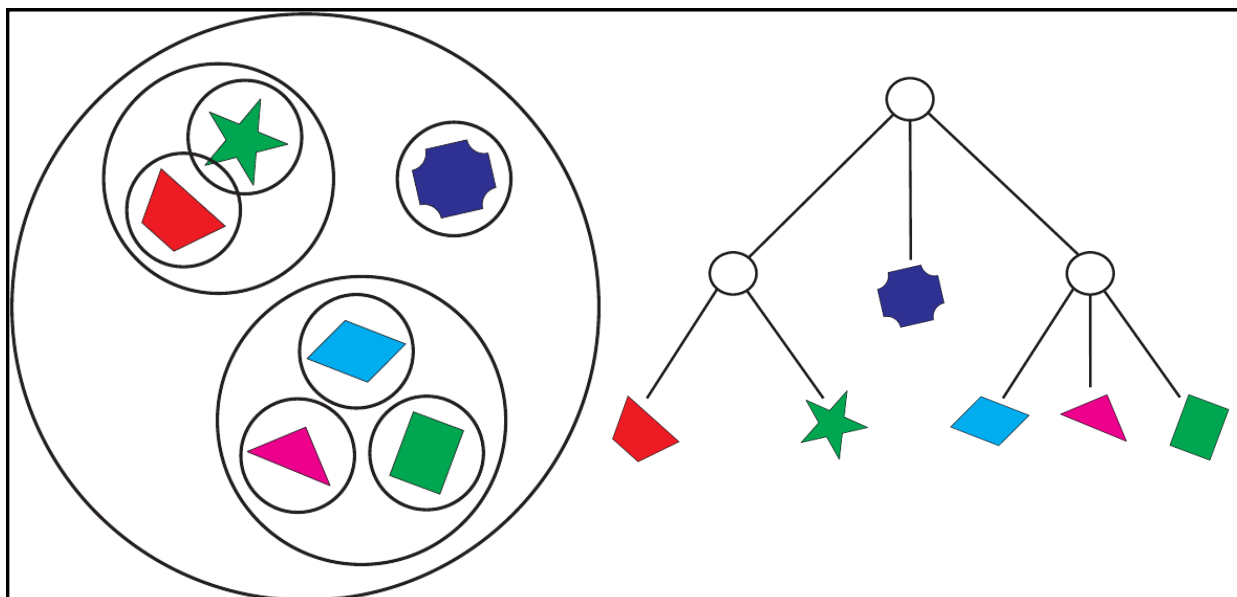
Fonte: Convex Hull- Applications. Disponível em http://www.tcs.fudan.edu.cn/rudolf/Courses/Algorithms/Alg_ss_07w/Webprojects/Chen_hull/applications.htm. Acesso em: 17/05/2014.



BVH

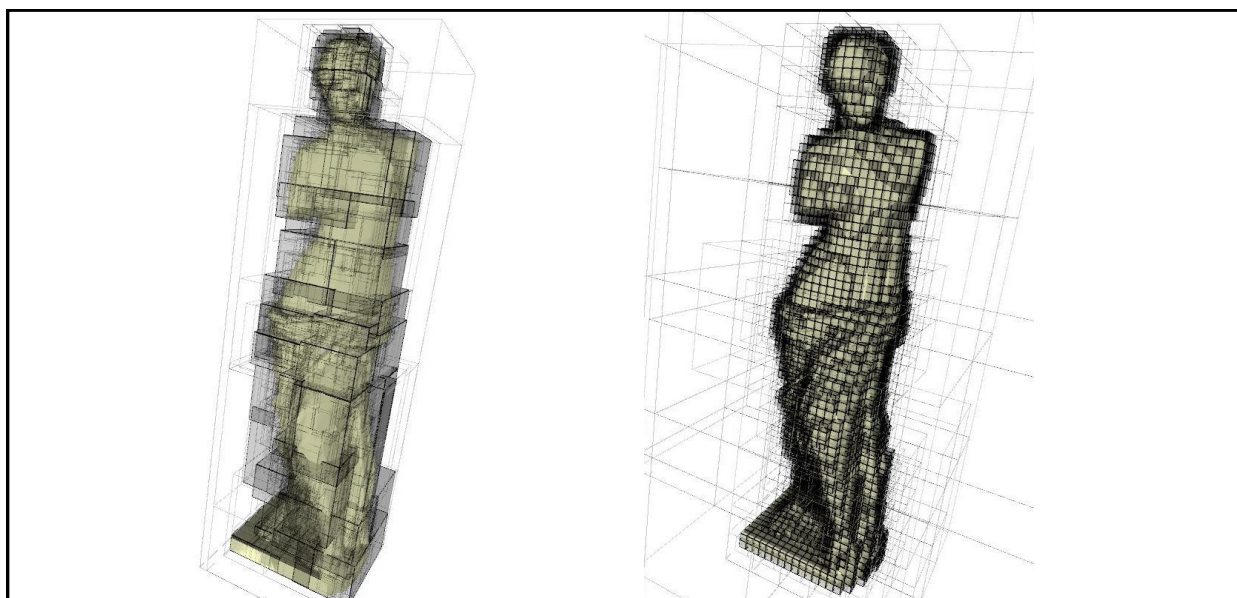
- ***Bounding Volume Hierarchies (BVH)***
 - Extensão de *Bounding Volumes* para acelerar buscas e testes de colisões/interseções
 - Organização em árvore
 - Nó pai contém todos os objetos dos nós filhos





Bounding Volumes Hierarchies

Fonte: (Helmrich 2009)



BVH (esquerda) vs. Octree (direita)

Fonte: Space-Partitioning: Octree vs. BVH. Disponível em: <http://thomasdiewald.com/blog/?p=1488>. Acesso em: 17/05/2014.



Pergunta

- Qual estrutura de dados devo usar?
 - Vai depender da sua aplicação
 - Pesquise diversos materiais na área e teste várias estruturas de dados
 - Essa apresentação não cobriu todas as estruturas de dados espaciais
 - Essas estruturas de dados ainda estão ganhando bastante atenção e pesquisa



Referências

- Shirley, P. *et al.* (2009). Fundamentals of Computer Graphics, Third Edition. A K Peters/CRC Press.
- Smith, A. R. (1995). A PIXEL IS NOT A LITTLE SQUARE, A PIXEL IS NOT A LITTLE SQUARE, A PIXEL IS NOT A LITTLE SQUARE, (AND A VOXEL IS NOT A LITTLE CUBE). *Tech Memo 6*.
- Amanatides, J., Woo, A. (1987). A Fast Voxel Traversal Algorithm for Ray Tracing. *Eurographics*.
- Samet, H. (1984). The Quadtree and Related Hierarchical Data Structures. *ACM Computing Surveys (CSUR)*.
- Brunet, P., Navazo, I. (1990). Solid representation and operation using extended octrees. *ACM Transaction on Graphics*.
- Kawashima, Y. *et al.* (1991). A flexible quantitative method for NC machining verification using a space-division based solid model. *The visual computer*.
- Helmrigh, H. (2009). Realtime Raytracing: Kd-Trees on graphics cards.

